Day 10 Learning(24/03/2026)

#GIT AND GITHUB:

1. Introduction to Git and GitHub

- **Git:** A distributed Version Control System (VCS) that runs locally on your computer. It tracks every change made to your code history.
- **GitHub:** A cloud-based hosting service for Git repositories. It allows developers to store their code online and collaborate with others.
- **Analogy:** If Git is your computer's "Save" button with a history log, GitHub is the "Google Drive" where you upload those saved files to share.

2. Key Concepts

- **Repository (Repo):** A folder where Git tracks all the project files and their history.
- **Local vs. Remote:** A Local Repo is on your machine; a Remote Repo is on a server like GitHub.
- **Working Directory:** The actual files you see and edit on your computer.
- **Staging Area (Index):** A "waiting room" where you add changes before committing them.
- **Commit:** A permanent snapshot of your changes in the history.

3. Advanced Operations (Branching and Merging)

- **Branching:** Creating a separate line of development to work on new features without affecting the main code.
- **Merging:** Combining two branches into one. It creates a "Merge Commit" and preserves the complete history of both branches.
- **Rebase:** Moving or combining a sequence of commits to a new base commit. It provides a clean, linear project history but rewrites history.
- **Cherry-pick:** The process of selecting a specific commit from one branch and applying it to another.

4. Special Features (Stash vs. Shelve)

- **Git Stash:** A Git command to temporarily store (hide) uncommitted changes so you can work on something else. It works at the Git level.
- **IntelliJ Shelve:** A feature specific to JetBrains IDEs (like IntelliJ). It allows you to put aside specific files or changes. It is managed by the IDE, not Git itself.

5. Collaboration Terms

- **Pull Request (PR) / Merge Request (MR):** A way to tell team members that you have finished a feature and want to merge it into the main branch. PR is used in GitHub; MR is used in GitLab.
- **Forking:** Creating a personal copy of someone else's project on your GitHub account to make changes independently.
- **Merge Conflict:** Occurs when Git cannot automatically resolve differences in code between two commits (usually when two people edit the same line).

6. Important Git Commands Table

Category	Command	Description
Setup	git init	Initialize a new local Git repository.
Setup	git clone [URL]	Download a project from GitHub to your local machine.
Basic Workflow	git add [file]	Move changes from Working Directory to Staging Area.
Basic Workflow	git commit -m "[msg]"	Save staged changes to the local repository history.
Basic Workflow	git status	View the state of your working directory and staging area.
Remote	git push	Upload local commits to the remote repository (GitHub).

Remote	git pull	Fetch and merge changes from remote to local.
Remote	git fetch	Download changes from remote without merging them.
Branching	git branch [name]	Create a new branch.
Branching	git checkout [name]	Switch to a different branch.
Merging	git merge [branch]	Merge the specified branch into the current branch.
Rebase	git rebase [branch]	Reapply commits on top of another base tip.
Temporary	git stash	Temporarily store all modified tracked files.
Temporary	git stash pop	Restore the most recently stashed files.
Specific	git cherry-pick [hash]	Apply the changes introduced by a specific commit.
Undo	git reset --hard HEAD	Discard all local changes and go back to the last commit.

7. Best Practices

- **Pull before you Push:** Always get the latest code from the remote repo to avoid conflicts.
- **Write Clear Commit Messages:** Explain "Why" you made the change, not just "What".
- **Use .gitignore:** Always include a .gitignore file to avoid tracking unnecessary files like .iml, .idea, or node_modules.

#DESIGN PATTERNS (CREATIONAL)

A. Singleton Design Pattern

- **Definition:** It ensures that a class has only one instance (object) in the entire application and provides a global point of access to it.
- **Why use it?** To save memory and resources. For example, you don't need 100 objects to connect to one Database; one shared object is enough.
- **How to implement:**
    1. Make the **Constructor private** (so no one can use the `new` keyword).
    2. Create a **static instance** of the same class.
    3. Provide a **public static method** (e.g., `getInstance()`) to return that instance.
- **Real-world Example:** A **Logger** or **Database Connection Pool**. There is only one Principal in a school, not one for every student.
- **Example code:**

```
public class DatabaseConnection {

    private static DatabaseConnection instance;

    private DatabaseConnection() {}

    public static DatabaseConnection getInstance() {

        if (instance == null) { instance = new DatabaseConnection(); }

            return instance;

    }

}
```

B. Factory Design Pattern

- **Definition:** It is used when we have a superclass with multiple sub-classes and based on input, we need to return one of the sub-classes. It hides the object creation logic.
- **Why use it?** It provides "Loose Coupling." The client doesn't need to know which specific class object is being created.
- **Real-world Example:** Consider a **Notification Factory**. If you pass "SMS", it returns an SMSNotification object. If you pass "Email", it returns an EmailNotification object.
- **Interview Tip:** Mention that it centralizes object creation, making the code easier to maintain.
- **Example code:**

```
public class NotificationFactory {

    public Notification createNotification(String type) {

        if (type.equals("SMS")) return new SMSNotification();

        if (type.equals("EMAIL")) return new EmailNotification();

        return null;

    }

}
```

// Client code:

// factory.createNotification("SMS"); // Hume internal logic se matlab nahi

C. Builder Design Pattern

- **Definition:** It is used to construct a complex object step-by-step. It solves the problem of having too many arguments in a constructor (Constructor Overloading).
- **Why use it?** If an object has 10 attributes, but only 3 are mandatory, the Builder pattern allows you to set only those 3 and "build" the object.
- **Real-world Example:** Ordering a **Pizza**. You choose the crust, then toppings, then extra cheese, and finally say "Create". You don't get a pre-made pizza where you have to remove things you don't like.
- **Example code:**

```
public class Laptop {
    private String ram;
    private String hdd;
    private boolean isGraphicsCardEnabled;
   // Builder class andar hi hoti hai
    public static class Builder {
        private Laptop laptop = new Laptop();
        public Builder setRam(String ram) {
            laptop.ram = ram;
            return this;
        }
        public Laptop build() {
            return laptop;
        }
    }
}
// Client code:
// Laptop myLaptop = new Laptop.Builder().setRam("16GB").build();
```

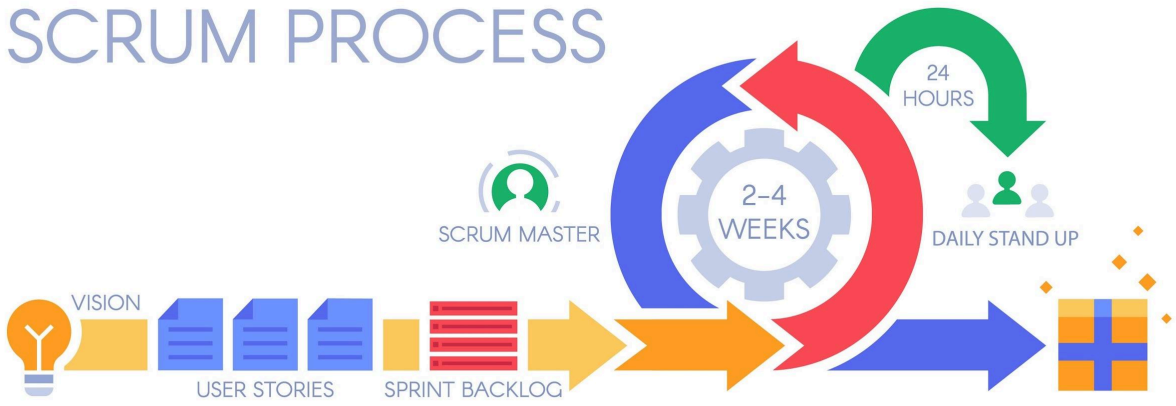
---

2. AGILE METHODOLOGY (SCRUM FRAMEWORK)

A. What is Agile?

- Agile is a mindset and a way of managing projects by breaking them into small, manageable chunks called **Increments**. It focuses on continuous feedback and rapid delivery.

- **Sprint:** A time-boxed period (usually **2 weeks**) where the team works to complete a set amount of work.



B. Key Roles in Scrum

- **Product Owner (PO):** Represents the customer. They maintain the **Product Backlog** and decide which features are the highest priority.
- **Scrum Master:** A facilitator who ensures the team follows Scrum values. They remove "Blockers" (obstacles) that stop the developers from working.
- **Development Team:** The group of professionals (Devs, QA, Designers) who build the product.

C. Important Agile Meetings (Rituals)

1. **Sprint Planning:** Held at the start of a Sprint. The team picks "User Stories" from the Product Backlog to work on during the next 2 weeks.
2. **Daily Scrum (Stand-up):** A 15-minute daily meeting. Every member answers:
    - What did I do yesterday?
    - What will I do today?
    - Are there any blockers (technical issues/doubts)?
3. **Effort Estimation Call (Grooming):**
    - The team discusses the **Complexity** of a task using **Story Points** (Fibonacci series: 1, 2, 3, 5, 8).
    - It's not about hours; it's about how "hard" or "risky" a task is.
4. **Sprint Review (Demo):** At the end of the Sprint, the team shows the finished work to the stakeholders/clients for feedback.
5. **Sprint Retrospective:** The most important meeting for improvement. The team discusses:

    - What went well?
    - What went wrong?
    - How can we improve in the next Sprint?

D. Key Artifacts (Tools)

- **User Story:** A requirement written from the user's perspective.
- **Product Backlog:** A master list of all features/bugs for the entire project.
- **Sprint Backlog:** The list of tasks the team agreed to finish in the current 2-week Sprint.
- **Burndown Chart:** A graph showing how much work is left versus time remaining in the Sprint.

#INTERVIEW KEY POINTS:

- **Difference between Scrum and Agile:** Agile is the philosophy/method; Scrum is one specific way to implement Agile.
- **Story Points vs. Hours:** We use Story Points because time is subjective (one dev might take 2 hours, another might take 5), but complexity remains the same for both.
- **What is a Blocker?** Anything that prevents you from coding (e.g., Environment down, unclear requirements, waiting for another team).